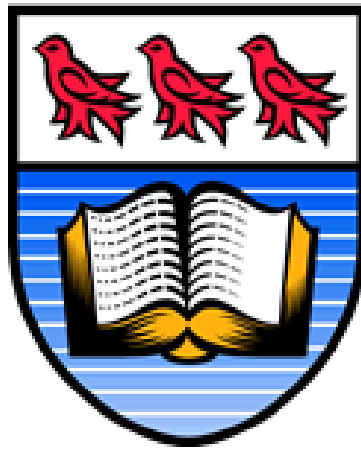


Department of Electrical and Computer
Engineering
University of Victoria



ECE 455 – Real Time Systems

Project 2: Deadline-Driven Scheduler

Submitted: April 4th, 2024

Lab: B02

Hoang Tuan Kiet Vu –

Brandon Radford -

Contents

List of Acronym.....	iii
List of Figures	iii
List of Tables:	iii
Introduction.....	1
System Architecture.....	2
Overview	2
FreeRTOS Tasks	3
DDS Task	3
Task Generator Task.....	3
User Task Functions	4
Monitor Task	5
Timers.....	5
Deadline Driven Scheduler Tasks	5
Request Handling	5
Task List Management	5
How it works	6
EDF Task Insertion Sorting Algorithm.....	7
Empty List Case.....	8
Head Comparison Case	8
Mid-List Insertion Case	8
End-of-List Case.....	9
System Testing and Evaluation.....	9
Discussion	11
Limitations and Possible Improvements	12
Conclusion	13
References.....	14
Appendix.....	14

List of Acronym

EDF: Earliest Deadline First

DDS: Deadline-Driven Scheduler

CPU: Central Processing Unit

List of Figures

Figure 1: The structure of the DD-task.	1
Figure 2: The definitions of real time tasks.	2
Figure 3: Overview Diagram of the System for DDS Implementation.	3
Figure 4: The flowchart of the DD task Generator Algorithm.	4
Figure 5: The flow of the Deadline-Driven Scheduler.	7
Figure 6: The flow of the DD Task Sorting Algorithm.	8
Figure 7: The test benches values.	9
Figure 8: The Excel bar graph of the EDF Schedulling for Test Bench 1.....	9
Figure 9: The Excel bar graph of the EDF Schedulling for Test Bench 2.....	10

List of Tables:

Table 1: The DDS events for test bench 1 over one hyper-period (1500ms).	10
Table 2: The output for the Monitor Task of test bench 2 over one hyper-period (1500ms)..	11

Introduction

The purpose of this project is to design and implement a dynamic, deadline-driven scheduler (DDS) for a FreeRTOS-based system on the STM32F4 Discovery board. This scheduler aims to manage a set of periodic tasks by applying the Earliest Deadline First (EDF) scheduling algorithm, a strategy known to be optimal for hard real-time systems when the task set is schedulable. Unlike fixed-priority scheduling, EDF prioritizes tasks based on their absolute deadlines at runtime.

Since FreeRTOS lacks built-in support for deadline-driven scheduling, this project required implementing a layer on top of FreeRTOS that dynamically adjusts task priorities to emulate EDF behaviour. All tasks are encapsulated in a custom data structure, and the system dynamically generates and manages them via a central DDS task. The project was tested against multiple test benches to validate the accuracy and efficiency of the scheduler, especially under high utilization or overloaded conditions.

According to the ECE455 Project 2 Lab Manual, the implementation must include:

- A custom DDS task that receives task release and completion events, maintains a dynamically sorted Active Task List by deadline, and manages task priority promotion/demotion.
- A Task Generator Task that periodically or aperiodically creates user-defined tasks (DD-Tasks).
- A set of User Task Functions that simulate task workloads with known execution times and signal completion.
- A Monitor Task that queries DDS for task list statuses and reports the number of Active, Completed, and Overdue tasks.
- Support for at least three test benches, including a schedulable, a borderline, and an overloaded task set.
- Usage of FreeRTOS Queues, Timers, and Semaphores to manage timing and inter-task communication.
- Compliance with hard real-time constraints: tasks that miss deadlines are dropped and reported.
- Proper memory management (e.g., dynamic list nodes, heap monitoring, freeing retired tasks).

Example of DDS can be seen below:

```
enum task_type {PERIODIC,APERIODIC};

struct dd_task {
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
}
```

Figure 1: The structure of the DD-task.

As part of the requirements, the deadline for each task will be treated as hard deadline to make sure all of the release time, absolute deadline, and completion time are accurate.

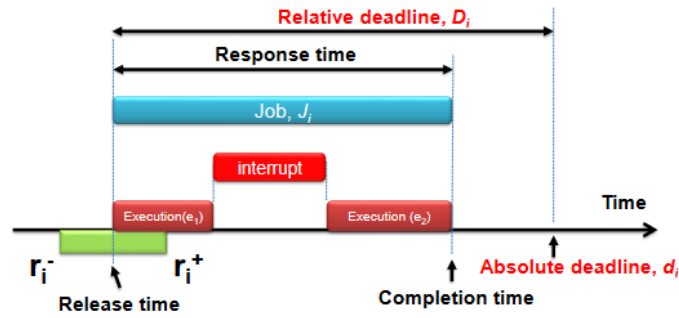


Figure 2: The definitions of real time tasks.

In the end, the performance and functionality of the program will be tested via three test benchmarks provided in the lab manual, all of which will be executed over 1500 ms. This part will be examined in depth in later sections.

System Architecture

Overview

The overall architecture follows a layered design where DDS operates as a high-priority FreeRTOS task managing lower-priority application tasks. The DDS task monitors a request queue to receive task events and performs the following responsibilities:

- Accepts release, complete, and overdue task events
- Maintains three linked lists of `ddTaskNode_t`: active, completed, and overdue tasks
- Uses EDF sorting to insert tasks into the active list
- Modifies task priorities to enforce EDF execution
- Uses a deadline timer to detect overdue tasks
- Responds to monitor queries for system status

All tasks and queues are dynamically created. The scheduler task (`deadlineSchedulerTask`) is given the highest priority (`DDS_PRIO = 3`), followed by auxiliary tasks (`AUX_PRIO = 2`) and user tasks (`RUN_PRIO = 1`). Paused tasks are assigned the lowest priority (`PAUSE_PRIO = 0`). The following diagram illustrates the interaction between all of the mandatory components of the system.

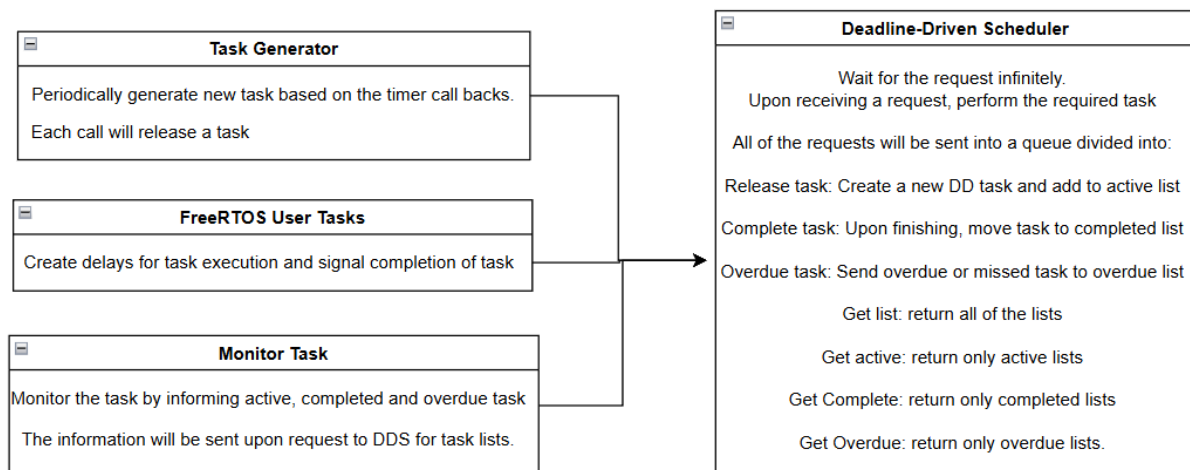


Figure 3: Overview Diagram of the System for DDS Implementation.

To simulate execution time, the FreeRTOS user tasks use delays and eventually call the `completeTask` function, which sends a completion request to the DDS. This mechanism is explained in more detail later in the report.

Meanwhile, the monitor task periodically requests updates from the DDS by calling functions to retrieve the active, completed, and overdue task lists. The DDS handles these requests by returning the appropriate lists, effectively acting as a centralized request handler.

In addition to the monitor task, the FreeRTOS tasks also send various requests to the DDS. These are added to a queue, and the DDS processes them based on their type and create new DD-tasks accordingly. Once created, these tasks are scheduled and tracked by the DDS.

FreeRTOS Tasks

DDS Task

- Executes with highest FreeRTOS priority (`DDS_PRIO = 3`).
- Processes messages from the request queue:
 - `RELEASE_EVENT`, `COMPLETE_EVENT`, `OVERDUE_EVENT`
 - `GET_LISTS`, `GET_ACTIVE`, `GET_COMPLETE`, `GET_OVERDUE`
- Manages three linked lists: Active, Completed, and Overdue.
- Performs EDF sorting during insertion of tasks into Active List.
- Uses `pauseScheduler()` and `updateScheduler()` to dynamically adjust task priorities and re-arm a deadline timer.

More detailed explanation will be provided after this section.

Task Generator Task

- Periodically triggered using `genTimer`.
- Tracks elapsed time for each task and releases them based on their configured periods:
 - Constructs `ddTask_t` objects.
 - Calls `releaseTask()` to send a `RELEASE_EVENT` to DDS.
- Runs with auxiliary priority (`AUX_PRIO = 2`).

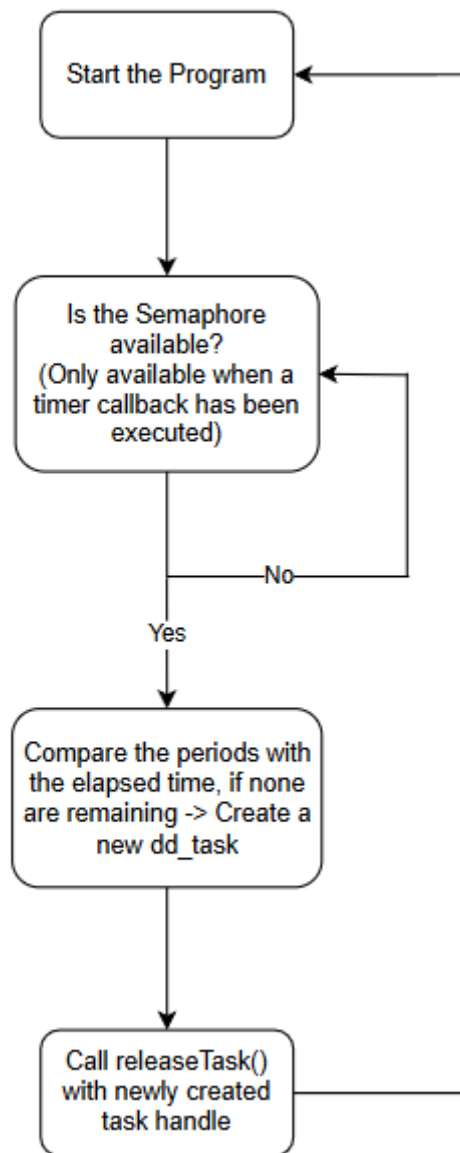


Figure 4: The flowchart of the DD task Generator Algorithm.

The task generator starts by setting the time counters for all three tasks to zero. It then checks whether a semaphore is available. If it is, the generator compares the current time with each task's period to see if enough time has passed. If the elapsed time is exactly divisible by the task's period, meaning there's no remainder, it creates a new DD-task with a fresh task handle. After that, it updates the time counter by adding the period value. Once that's done, the generator loops back to check the semaphore again and repeats the process.

User Task Functions

- Three user tasks (userTask1, userTask2, userTask3).
- Each performs a busy wait using delayMS() for a predefined duration (simulating execution).
- On completion, calls completeTask() to inform the DDS.

- Created dynamically by the DDS during insertion into Active List.
- Task priority is managed exclusively by DDS (PAUSE_PRIO or RUN_PRIO).

Monitor Task

- Triggered periodically via monitorTimer.
- Uses getActiveList(), getCompleteList(), and getOverdueList() to inspect DDS task lists.
- Counts and reports list sizes with printf.
- Runs with AUX_PRIO = 2, lower than DDS.

The monitor task is responsible for collecting and printing the number of tasks in the active, completed, and overdue lists to the console. To do this, it first sets up list pointers and counters for each type of task. It then waits for the monitor timer to expire, handled using a semaphore. Once the timer triggers, the monitor sends requests to the DDS to retrieve the current state of each list. It then loops through each list, counts the number of tasks, and, if monitoring is enabled, prints the current time along with the counts for each task list.

Timers

- sysTimer: Updates a global millisecond time counter.
- genTimer: Periodically releases user tasks via semRelease.
- monitorTimer: Triggers monitor via semUpdate.
- deadlineTimer: One-shot timer, armed with the deadline of the current running task. Sends OVERDUE_EVENT if deadline missed.

Deadline Driven Scheduler Tasks

Request Handling

The deadlineSchedulerTask() waits on requestQueue, processing each received DDSMessage_t. The DDS supports the following request types:

- **RELEASE_EVENT**: Inserts a new task into the active list using insertTask()
- **COMPLETE_EVENT**: Removes the currently running task and places it in the completed list
- **OVERDUE_EVENT**: Removes the head task and adds it to the overdue list
- **GET_ACTIVE, GET_COMPLETE, GET_OVERDUE, GET_LISTS**: Responds by sending appropriate list(s) to outputQueue

Before handling any event, the currently active task is paused (priority lowered), and after processing, updateScheduler() is called to resume the new highest-priority task.

Task List Management

- **insertTask()**: Inserts a task into the active list in sorted order based on absDeadline. It dynamically allocates memory using pvPortMalloc and creates the FreeRTOS task via xTaskCreate().
- **removeTask()**: Removes the head of the active list and transfers it to the appropriate retired list (completed or overdue). The task is deleted using vTaskDelete(), and its completion time is recorded.

- **pauseScheduler():** Temporarily demotes the running task's priority to PAUSE_PRIO to allow scheduling changes.
- **updateScheduler():** Promotes the head of the active list to RUN_PRIO, and sets deadlineTimer to fire at the task's absolute deadline.

How it works

The DDS starts by waiting in an idle state until it receives a request. Once a request comes in, it pauses all currently active tasks to determine what type of request it is. If the request is for a task list, the DDS responds by sending the appropriate list, whether it's the active, completed, or overdue tasks. If the request is to release a task, the scheduler creates a new DD-task and calls the updateScheduler() function to determine where the task should be inserted in the schedule. This scheduling process is discussed in more detail later in the report. The same general process applies when a task deletion request is received, using the removeTask() function. After handling the request, the DDS returns to its waiting state, ready for the next one.

Each DD task in the system is defined using a structure that includes all the key information needed to manage it: the deadline, execution time, current state, and its corresponding FreeRTOS task handle. This structure helps keep everything organized and makes it easier to track tasks from start to finish. By grouping this data together, the system maintains a clear connection between the task's properties and the actual FreeRTOS task that runs it.

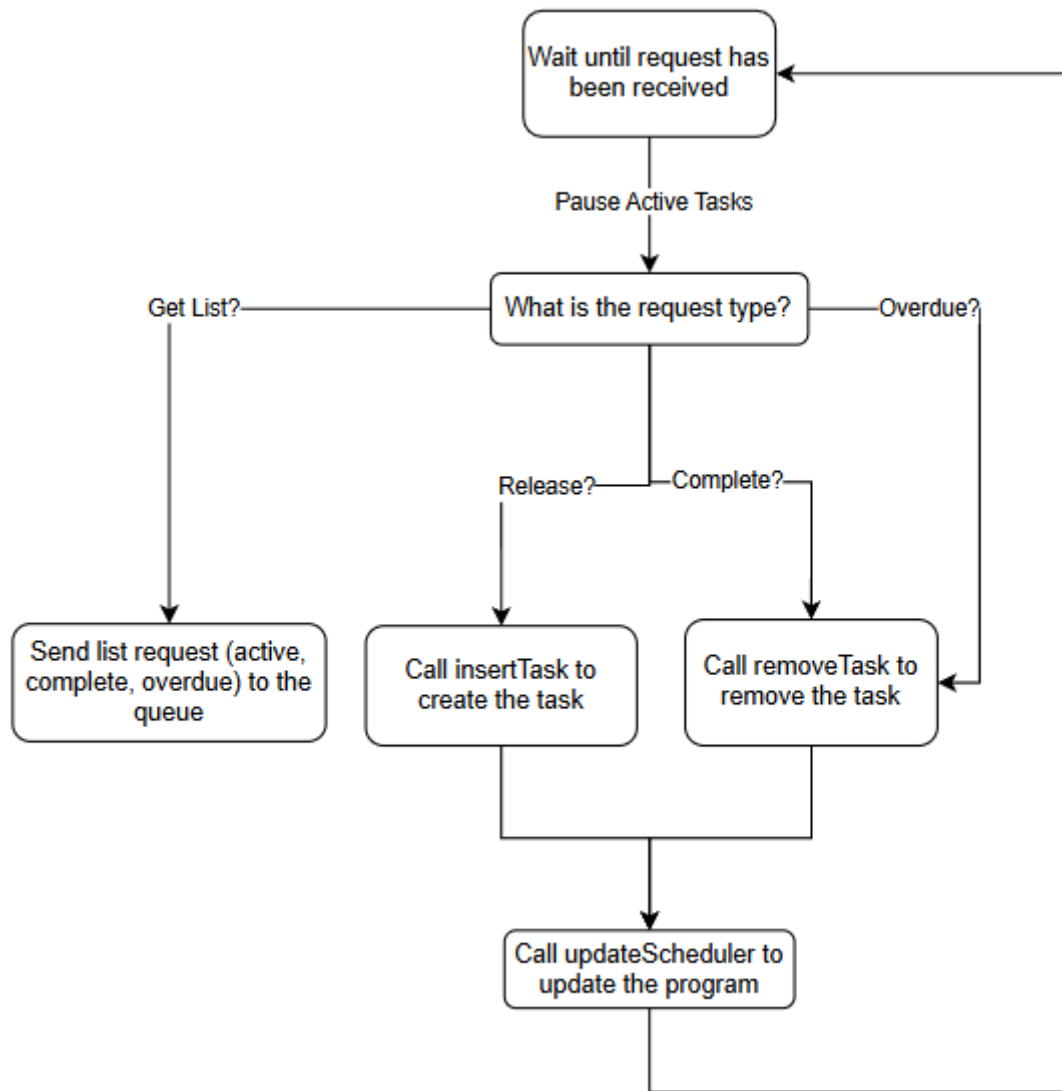


Figure 5: The flow of the Deadline-Driven Scheduler.

EDF Task Insertion Sorting Algorithm

The EDF scheduling policy requires that all active tasks be ordered by their absolute deadlines so that the task with the earliest deadline is always at the head of the list. The DDS inserts each newly released task into the active task list. The following cases could be derived from the algorithm:

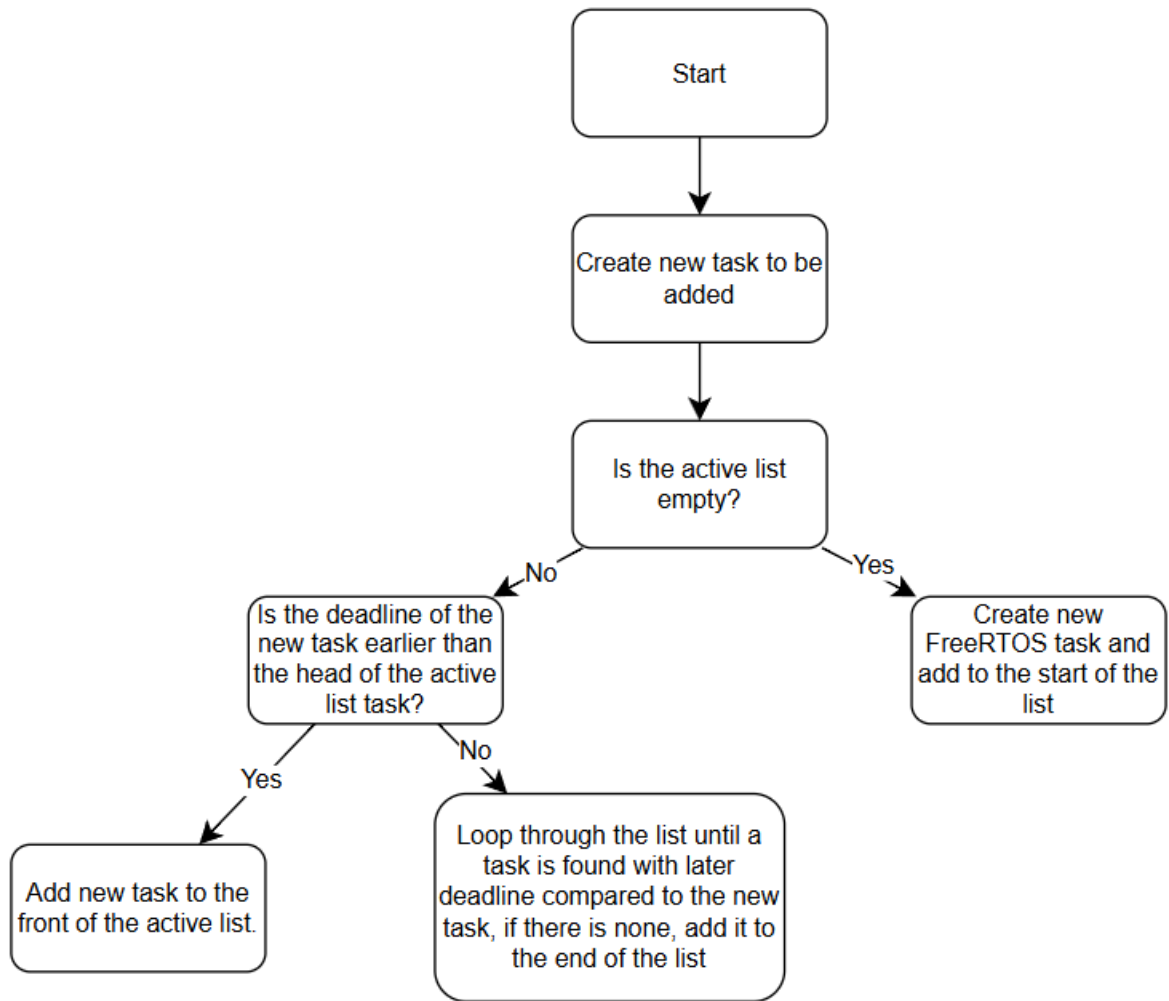


Figure 6: The flow of the DD Task Sorting Algorithm.

Empty List Case

When a new task is released and the active list is currently empty, the task is inserted directly as the head of the list. This is the simplest case, as no ordering checks are necessary. A new task node is dynamically allocated, and the associated FreeRTOS task is created and assigned to the node.

Head Comparison Case

If the active list is not empty, the scheduler compares the absolute deadline of the new task with that of the current head node.

- If the new task has an earlier deadline than the head, it is inserted at the front of the list.
- This preserves EDF ordering since the head of the list must always have the earliest deadline.

Mid-List Insertion Case

If the new task's deadline is not earlier than the head, the DDS iterates through the active list.

- The scheduler searches for the **first node** whose deadline is later than the new task's deadline.

- Once this insertion point is found, the new task is inserted **before** the task with the later deadline.

End-of-List Case

If no such insertion point is found, all existing tasks have earlier or equal deadlines, the new task is appended to the end of the list. Throughout this process, memory for the new task node is allocated using `pvPortMalloc()` and the FreeRTOS task is instantiated using `xTaskCreate()`.

System Testing and Evaluation

To evaluate the functionality and real-time performance of the program, three predefined test benches were executed. The parameters for each test bench were derived directly from the ECE 455 Project 2 lab manual [1]:

Task	Test Bench #1		Test Bench #2		Test Bench #3	
	Execution Time (ms)	Period (ms)	Execution Time (ms)	Period (ms)	Execution Time (ms)	Period (ms)
t_1	95	500	95	250	100	500
t_2	150	500	150	500	200	500
t_3	250	750	250	750	200	500

Figure 7: The test benches values.

Before executing the tests on hardware, the team manually sketched the expected scheduling behavior for Test Bench 1 and Test Bench 2 using Excel-based Gantt-style diagrams. These diagrams served as references to validate the actual run-time output generated by the DDS implementation. Each test was conducted over one hyper-period of 1500 milliseconds, and task activity was logged and compared against the expected outcomes.

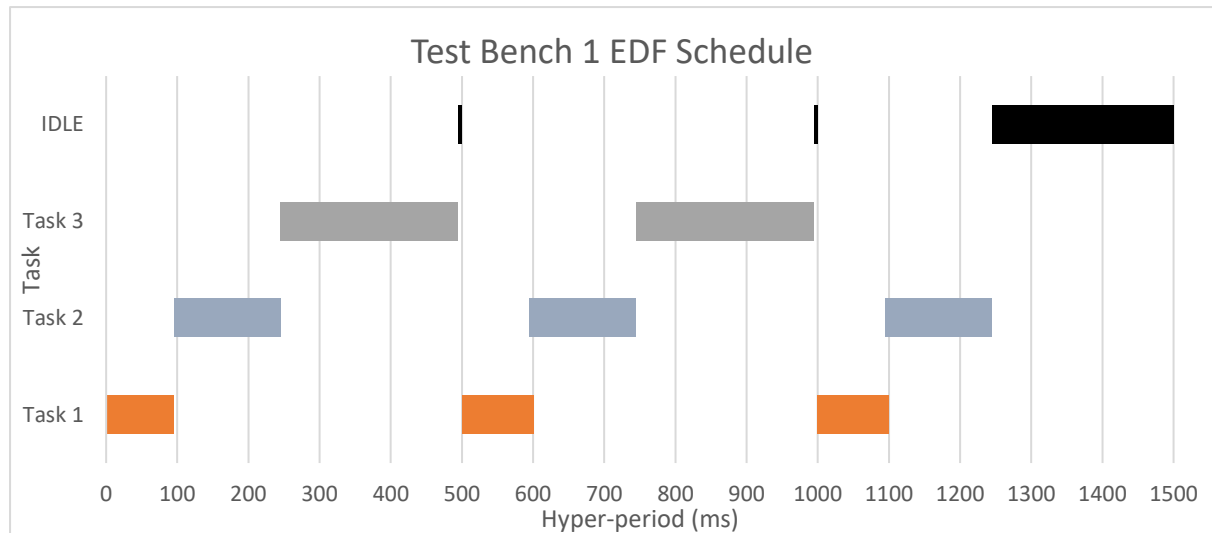


Figure 8: The Excel bar graph of the EDF Scheduling for Test Bench 1.

Figure 15 illustrates the expected execution order for Test Bench 1 under EDF scheduling, where Task 1 (T1), Task 2 (T2), and Task 3 (T3) have execution times of 95 ms, 150 ms, and 250 ms, and periods of 500 ms, 500 ms, and 750 ms respectively. During a single hyper-period of 1500 ms, the EDF algorithm prioritizes tasks based on the earliest absolute

deadline, resulting in a sequence where T1 and T2 frequently preempt T3 due to their shorter periods. At the end of the hyper-period, no tasks remained active, indicating that all scheduled tasks had been processed, either successfully or as overdue, validating the EDF policy's correctness under a balanced load scenario.

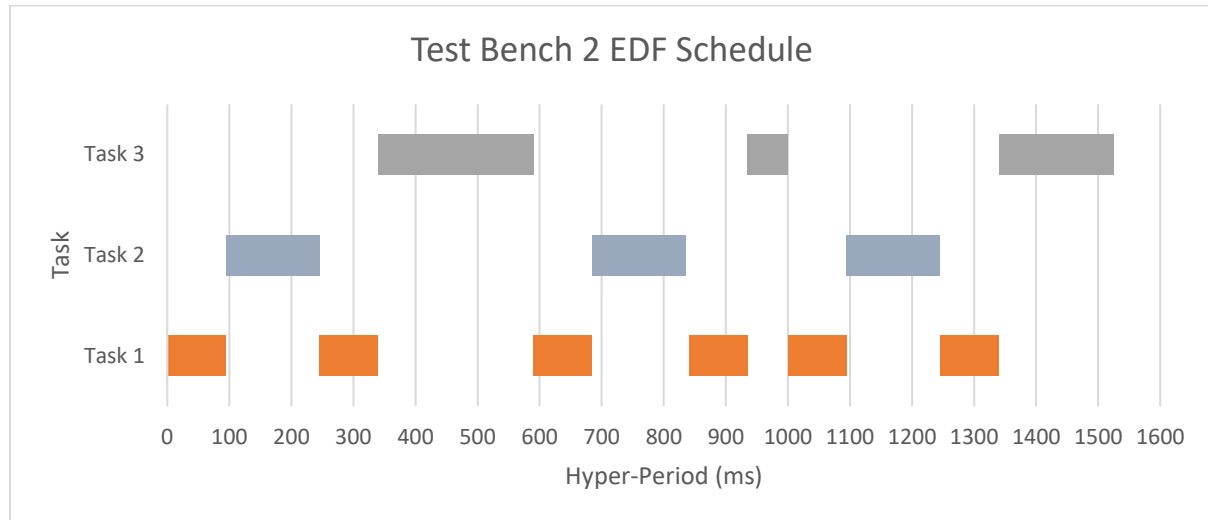


Figure 9: The Excel bar graph of the EDF Scheduling for Test Bench 2.

Figure 16 illustrates the expected EDF scheduling behavior for Test Bench 2, where Task 1 (T1), Task 2 (T2), and Task 3 (T3) have execution times of 95 ms, 150 ms, and 250 ms, and periods of 250 ms, 500 ms, and 750 ms respectively. Due to T1's shorter period, tasks are released more frequently, increasing system load and tightening scheduling constraints. Over a 1500 ms hyper-period, the EDF schedule predicts that ten tasks would complete on time, one task would miss its deadline and be classified as overdue, and three tasks would still be active. By 1600 ms, these values remained consistent, confirming the expected result. The presence of an overdue task highlights the system's high utilization and confirms that while EDF scheduling remains effective under pressure, limited processor availability and overhead can lead to occasional deadline misses, especially in more demanding workloads.

While Test Bench 3 was not visualized through a similar pre-simulation diagram, its behavior was well-understood from a theoretical standpoint. Specifically, this test case was designed with a worst-case processor utilization of 100%, meaning the CPU was fully occupied with task execution. This left no slack time for the DDS overhead, context switching, or any auxiliary operations. Consequently, during Test Bench 3 execution, some task deadlines were unavoidably missed, as the processor had insufficient headroom to handle all scheduled work and system overhead simultaneously.

After completing the calculations, the program was ran using the hardware. The following result was retrieved:

Table 1: The DDS events for test bench 1 over one hyper-period (1500ms).

Event Number	Event	Measured Time (ms)	Expected Time (ms)
1	T1 Released	0	0
2	T2 Released	0	0
3	T3 Released	0	0

4	T1 Completed	95	95
5	T2 Completed	247	245
6	T3 Completed	499	495
7	T1 Released	500	500
8	T2 Released	500	500
9	T1 Completed	596	595
10	T2 Completed	748	745
11	T3 Released	750	750
12	T3 Completed	1003	1000
13	T1 Released	1000	1000
14	T2 Released	1000	1000
15	T1 Completed	1098	1095
16	T2 Completed	1250	1245
17	T1 Released	1500	1500
18	T2 Released	1500	1500
19	T3 Released	1500	1500

It can be seen the outcomes of Test Bench 1 were closely aligned with the theoretical expectations illustrated in the excel sheet. The observed release and completion times for all task instances matched the expected values with acceptable. Although there was some delay, however, this minor deviation is likely attributed to system overhead introduced by the monitoring task, specifically the use of the printf and other auxiliary functions. Despite this, the scheduler consistently executed tasks in the correct EDF order, confirming both the accuracy and real-time responsiveness of the implementation.

Table 2: The output for the Monitor Task of test bench 2 over one hyper-period (1500ms).

Task List	Measure	Expected
Number of Active DD-Tasks	3	3
Number of Completed DD-Tasks	10	10
Number of Overdue DD-Tasks	1	1

From the results for Test Bench 2, in comparison with the ones calculated in the sheet, it can be seen that the expected and measured values of active, completed and overdue DD-tasks match perfectly, indicating that the EDF DDS schedule was functioning as it was intended to.

Discussion

During the implementation of the DDS system outlined in this report, the project objectives were successfully met through a methodical design and development process. Similar to Project 1, Project 2 demanded a significant amount of time and effort outside of scheduled lab sessions, primarily due to the system's complexity and the number of design factors that needed to be considered. What distinguished this project from previous coursework was the considerable emphasis on design planning before implementation. The time invested in carefully mapping out the scheduler architecture and logic was nearly equivalent to the time

spent writing and debugging the code. This disciplined approach to design ultimately paid off, allowing the system to be implemented efficiently and correctly.

Although many core concepts were introduced in Project 1, Project 2 was substantially more challenging due to the added complexity of EDF scheduling, dynamic task management, and multi-layered task interactions. Fortunately, the absence of hardware integration in this project simplified testing and debugging, allowing our team to concentrate entirely on software development.

Despite careful planning, two significant software issues emerged during development. First of all, dynamic memory allocation was extremely difficult to implement as multiple different tasks were running simultaneously. Initially, FreeRTOS was configured to use heap_1, which does not support memory deallocation. This led to memory exhaustion when task nodes were dynamically allocated and never freed. Following guidance from the lab manual, the memory allocator was switched to heap_2, which supports both allocation and deallocation, resolving the issue. The second challenge was related to the algorithm itself, as it was easily mistaken as a different approach. Finding a way to control all aspects of the program promptly was also proven to be quite challenging.

Project 2 served as a valuable and practical learning experience. Project 0 provided an initial introduction to FreeRTOS concepts, while Project 1 demonstrated how the FreeRTOS interfaces with hardware peripherals. Project 2, however, focused entirely on the software architecture and scheduling mechanisms, enabling a deeper exploration of FreeRTOS task behaviour and real-time scheduling policies. Concepts like EDF scheduling, priority management, dynamic list handling, and timer-based callbacks were entirely new, making this project a meaningful opportunity to apply theory to real-world embedded design.

Limitations and Possible Improvements

While the DDS implementation fulfilled all project requirements, several limitations were identified during testing and performance analysis:

- **Deadline Violation Detection Latency:** The scheduler only detects overdue tasks after their deadlines expire. It does not proactively prevent tasks from running late, which could be addressed through predictive deadline monitoring or slack-time analysis.
- **Task List Sorting Overhead:** Every task release requires the active list to be traversed and re-sorted by deadline. This incurs linear time complexity and may become a performance bottleneck with higher task volumes.
- **Rigid Deadline Assumptions:** The system assumes every task has a strict absolute deadline. In real systems, many background or lower-priority tasks may not require such constraints, leading to unnecessary overhead when assigning arbitrary distant deadlines.
- **32-bit Timer Overflow:** The system time is tracked using a 32-bit counter, which overflows after approximately 49.7 days of continuous operation. While not a problem in lab settings, this could affect long-term deployments. Mitigation strategies include using a Real-Time Clock (RTC) or implementing a rollover-safe time management system.

- **Monitoring Overhead:** The monitorTask introduces system inefficiency by sending multiple queue requests to fetch active, completed, and overdue lists. It also iterates through each list on every cycle and prints statistics using printf(), a blocking and resource-intensive function in embedded systems.

To address these limitations, several enhancements are recommended:

- Implement a running counter for each list that is updated incrementally, rather than traversing the entire list during monitoring.
- Explore alternative scheduling algorithms such as Least Slack Time (LST), First-In-First-Out (FIFO), or hybrid EDF+LST models, which may reduce sorting overhead or better handle aperiodic tasks.
- Replace printf() with a more efficient output method, such as UART logging, I2C LCD output, or LED indicators for monitoring task counts.
- Periodically free memory from completed and overdue lists to minimize heap fragmentation.
- Introduce deadline monitoring preemption logic to identify when a task is unlikely to meet its deadline and re-prioritize accordingly.

Collectively, these improvements would enhance both the real-time reliability and the resource efficiency of the scheduler. Moreover, it was also mentioned by the lab TA that the implemented design might not be the right approach. As it seems in our logic, the program first executes the task and then starts the scheduling right after. Although in the context of the lab and the complexity of the design, this approach was not an issue as seen from the obtained results. However, this is a crucial mistake if made in real-world problems.

Conclusion

In conclusion, Project 2 required the design and implementation of a deadline-driven scheduler using EDF scheduling on FreeRTOS. The system was constructed using dynamically managed task structures, a central DDS controller, a periodic task generator, and a monitor task. Three independent test benches were used to evaluate performance, with each confirming the scheduler's correctness under both balanced and high-utilization scenarios.

Despite some development challenges, such as memory allocator limitations, heap management and more, these issues were resolved through careful debugging and configuration adjustments. The project emphasized the importance of both pre-design and systematic testing. By engaging with low-level RTOS operations, real-time scheduling algorithms, and dynamic memory management, the team deepened its understanding of embedded systems design.

Furthermore, the collaboration between group members and peers in the lab environment was integral to the project's success. The hands-on experience reinforced lecture concepts and bridged the gap between theoretical understanding and practical implementation.

Overall, this project was both demanding and rewarding, offering valuable insights into real-time system design. The team would like to express sincere gratitude to our teaching assistants, **LeHung** and **Pouya Shiri**, for their continuous support and guidance throughout the course of the project.

References

[1] B. Harrison, *ECE 455 Lab Manual*. University of Victoria, Dept. of Electrical and Computer Engineering, Mar. 2025, <https://www.ece.uvic.ca/~brent/ece455/ECE455-Lab-Manual.pdf>

Appendix

Below will be the screenshots from the program.

```
// Hoang Tuan Kiet Vu - XXXXXXXXXX

#include <stdint.h>
#include <stdio.h>
#include "stm32f4_discovery.h"

// FreeRTOS includes
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

//-----
// Configuration Macros
//-----
#define INCLUDE_vTaskSuspend      1 // logic macro definition
#define INCLUDE_vTaskDelete      1
#define configUSE_TIMERS         1
#define configSUPPORT_DYNAMIC_ALLOCATION 1

#define QUEUE_LEN                10
#define PAUSE_PRIO               0 // Lowest priority for paused
tasks
#define RUN_PRIO                 1 // Running task's priority
#define AUX_PRIO                 2 // Auxiliary tasks (monitor,
generator)
#define DDS_PRIO                 3 // Highest priority for DDS task

#define MONITOR_INTERVAL         100 // Monitor period in ms

// For Test bench 1
#define TASK_GEN_INTERVAL        250 // Task generator period in ms

// Task periods and execution times (in milliseconds)
#define TASK1_EXEC               95
#define TASK1_PERIOD             500
```

```

#define TASK2_EXEC                150
#define TASK2_PERIOD              500

#define TASK3_EXEC                250
#define TASK3_PERIOD              750

//// For Test bench 2
// #define TASK_GEN_INTERVAL      250    // Task generator period in ms
//
//// Task periods and execution times (in milliseconds)
// #define TASK1_EXEC             95
// #define TASK1_PERIOD           200
//
// #define TASK2_EXEC             150
// #define TASK2_PERIOD           500
//
// #define TASK3_EXEC             250
// #define TASK3_PERIOD           750

//// For Test bench 3
// #define TASK_GEN_INTERVAL      500    // Task generator period in ms
//
//// Task periods and execution times (in milliseconds)
// #define TASK1_EXEC             100
// #define TASK1_PERIOD           500
//
// #define TASK2_EXEC             200
// #define TASK2_PERIOD           500
//
// #define TASK3_EXEC             200
// #define TASK3_PERIOD           500

//-----
// Data Structures
//-----
typedef enum {
    PERIODIC,
    APERIODIC
} taskType;

typedef struct ddTask {
    TaskHandle_t    taskHandle;    // FreeRTOS task handle
    TaskFunction_t  taskFunc;      // Pointer to task function
    taskType        type;          // PERIODIC or APERIODIC
    char            *taskID;       // Task identifier (e.g., "T1")
    uint32_t        taskNum;       // Numeric identifier
    uint32_t        releaseTime;   // Release time (ms)

```

```

    uint32_t      absDeadline;    // Absolute deadline (ms)
    uint32_t      completionTime; // Completion time (ms)
} ddTask_t;

typedef struct ddTaskNode {
    ddTask_t      task;           // DD-Task data
    struct ddTaskNode *next;      // Pointer to next node in the list
} ddTaskNode_t;

typedef enum {
    RELEASE_EVENT, // Request to release a task
    COMPLETE_EVENT, // Request to complete a task
    OVERDUE_EVENT, // Request to mark a task overdue
    GET_LISTS,     // Request to get all task lists
    GET_ACTIVE,    // Request to get active list
    GET_COMPLETE,  // Request to get complete list
    GET_OVERDUE    // Request to get overdue list
} EventType;

typedef struct {
    EventType reqType; // Type of request
    ddTask_t task;     // Relevant task data (if applicable)
} DDSMessage_t;

//-----
// Global Variables (Queues, Semaphores, Timers)
//-----
xQueueHandle requestQueue = 0; // Queue for sending DDS messages
xQueueHandle outputQueue = 0;  // Queue for returning task lists

SemaphoreHandle_t semRelease = 0; // Semaphore to trigger task release
SemaphoreHandle_t semUpdate = 0;  // Semaphore to trigger monitor update

TimerHandle_t sysTimer = 0; // Timer for system timekeeping
TimerHandle_t deadlineTimer = 0; // Timer for checking deadlines
TimerHandle_t genTimer = 0; // Timer for triggering task generation
TimerHandle_t monitorTimer = 0; // Timer for triggering monitor task

//-----
// Function Prototypes
//-----
static void setupHardware(void);
void delayMS(uint32_t ms);
uint32_t getCurrentTime(void);

// DDS Interface Functions
BaseType_t releaseTask(ddTask_t task);
BaseType_t completeTask(void);

```

```

void getTaskLists(ddTaskNode_t **active, ddTaskNode_t **overdue, ddTaskNode_t
**complete);
void getActiveList(ddTaskNode_t **active);
void getCompleteList(ddTaskNode_t **complete);
void getOverdueList(ddTaskNode_t **overdue);

// Task List Management
BaseType_t insertTask(ddTaskNode_t **activeList, ddTask_t newTask);
BaseType_t removeTask(ddTaskNode_t **activeList, ddTaskNode_t **retiredList);
BaseType_t pauseScheduler(ddTaskNode_t **activeList);
BaseType_t updateScheduler(ddTaskNode_t **activeList);

// FreeRTOS Tasks
void deadlineSchedulerTask(void *pvParameters);
void taskGeneratorTask(void *pvParameters);
void monitorTask(void *pvParameters);

// User Task Functions
void userTask1(void *pvParameters);
void userTask2(void *pvParameters);
void userTask3(void *pvParameters);

// Timer Callback Functions
void deadlineTimerCallback(TimerHandle_t xTimer);
void systemTimerCallback(TimerHandle_t xTimer);
void referenceTimerCallback(TimerHandle_t xTimer);

//-----
// Main Function Overall execution
/*
    taskGeneratorTask() builds a ddTask_t with userTask1 as its function.

    It sends that to DDS via releaseTask().

    deadlineSchedulerTask() receives it.

    Calls insertTask() → creates the task with priority 0.

    Then updateScheduler() → raises priority to 1. and RUN the code

    After that go to userTask1 and actually start the task

    based on the active, complete, overdue, the program check for the next course
    of action, informing DDS accordingly

    Every 250ms, the generator checks for task releases.

    DDS keeps scheduling tasks in earliest-deadline-first order.

```

User tasks keep simulating work and either complete or overrun.

Monitor keeps reporting system status every 100ms.

DDS keeps all task states updated dynamically.

```
*/
//-----
int main(void) {
    setupHardware();

    // Create queues for DDS messages and outputs - Initializes interrupt
    controller
    requestQueue = xQueueCreate(Queue_LEN, sizeof(DDSMessage_t)); // main DDS
    event queue (task release, completion, overdue).
    outputQueue = xQueueCreate(Queue_LEN, sizeof(ddTaskNode_t **)); // used
    to send active/complete/overdue lists back to monitor.

    // Create core DDS tasks
    xTaskCreate(deadlineSchedulerTask, "DDS", 5000, NULL, DDS_PRIO, NULL); //
    create central scheduler logic (highest priority).
    xTaskCreate(monitorTask, "MONITOR", configMINIMAL_STACK_SIZE, NULL,
    AUX_PRIO, NULL); // create prints task stats.
    xTaskCreate(taskGeneratorTask, "GEN", configMINIMAL_STACK_SIZE, NULL,
    AUX_PRIO, NULL); // generates periodic tasks.

    // Create semaphores for synchronization - used to trigger task actions
    from timer callbacks
    semRelease = xSemaphoreCreateBinary(); // For taskGenerator
    semUpdate = xSemaphoreCreateBinary(); // for monitorTask

    // Create and start system timers
    sysTimer = xTimerCreate("SysTimer", 1 / portTICK_PERIOD_MS, pdTRUE, (void
    *)0, systemTimerCallback);
    //1ms periodic timer. Increments internal time counter (TimerID).
    xTimerStart(sysTimer, 0);

    // dynamically adjusted for the current task's deadline.
    deadlineTimer = xTimerCreate("DeadlineTimer", 10000 / portTICK_PERIOD_MS,
    pdFALSE, (void *)1, deadlineTimerCallback);
    xTimerStart(deadlineTimer, 0);

    // Fires periodically to trigger task generation.
    genTimer = xTimerCreate("GenTimer", pdMS_TO_TICKS(TASK_GEN_INTERVAL),
    pdTRUE, (void *)2, referenceTimerCallback);
    xTimerStart(genTimer, 0);
```

```

    // Triggers monitor task to print status.
    monitorTimer = xTimerCreate("MonitorTimer",
pdMS_TO_TICKS(MONITOR_INTERVAL), pdTRUE, (void *)3, referenceTimerCallback);
    xTimerStart(monitorTimer, 0);

    vTaskStartScheduler(); // Start FreeRTOS

    /*
    The three created tasks are now running:

    deadlineSchedulerTask() (priority 3 – highest)

    monitorTask() (priority 2)

    taskGeneratorTask() (priority 2)

    Since DDS_PRIO = 3, the scheduler immediately runs:
    */
    while (1); // Infinite loop (never reached unless scheduler fails).
    return 0;
}

//-----
/* RUN FIRST Deadline Scheduler Task Central DDS logic (receives task
release/completion events, manages task lists).

Listens on requestQueue for events (release, complete, overdue).

Handles:

RELEASE_EVENT → insert new task in the active list (sorted by deadline).

COMPLETE_EVENT → move head task to complete list.

OVERDUE_EVENT → move head task to overdue list if past deadline.

Maintains 3 task lists: activeList, completeList, overdueList

Pauses the currently running task before any change using pauseScheduler.

Updates the scheduler (updateScheduler) to set the priority and deadline
timer.

*/

//-----
void deadlineSchedulerTask(void *pvParameters) {
    BaseType_t status; // to check if the queue give any message

```

```

    ddTaskNode_t *activeList = NULL; // linkedlist for currently running or
ready to run task
    ddTaskNode_t *overdueList = NULL; // linkedlist for tasks that missed the
deadline
    ddTaskNode_t *completeList = NULL; // Tasks hat finished
    uint32_t currentTime = 0;

    for (;;) {
        DDSMessage_t msg;
        status = xQueueReceive(requestQueue, &msg, portMAX_DELAY); // wait for
the message, store value in msg than wait
        // here until requestQueue receive a message -> go to
taskGeneratorTask() and trigger DDS to release event -> releaseTask
        currentTime = getCurrentTime();

        // after release event 1 got trigger, check the status and proceed
        if (status) {
            EventType event = msg.reqType; // tells us what kind of message
it is.

            ddTask_t receivedTask = msg.task; // has all the task details

            if (event >= GET_LISTS) {
                // Handle status requests
                switch (event) {
                    case GET_LISTS:
                        xQueueSend(outputQueue, &activeList, 0);
                        xQueueSend(outputQueue, &completeList, 0);
                        xQueueSend(outputQueue, &overdueList, 0);
                        break;
                    case GET_ACTIVE:
                        xQueueSend(outputQueue, &activeList, 0);
                        break;
                    case GET_COMPLETE:
                        xQueueSend(outputQueue, &completeList, 0);
                        break;
                    case GET_OVERDUE:
                        xQueueSend(outputQueue, &overdueList, 0);
                        break;
                    default:
                        break;
                }
            } else {
                // Before processing an event, pause the current active task
                /*
                If we already had a running task, we lower its priority.

                So it won't interrupt us while we decide who should run next.

```

```

        */
        pauseScheduler(&activeList);
        switch (event) {
            case RELEASE_EVENT: // go first
                insertTask(&activeList, receivedTask); // Insert new
task into active list
                break;
            case COMPLETE_EVENT: // go second, after task finished
                removeTask(&activeList, &completeList); // after that
remove the finished task from the active list
                break;
            case OVERDUE_EVENT:
                if (activeList && (activeList->task.absDeadline <
getCurrentTime()))
                    removeTask(&activeList, &overdueList);
                break;
            default:
                break;
        }
        updateScheduler(&activeList); // go to updateScheduler
    }
}

// Memory management: if heap is low, free nodes from completeList
if (xPortGetFreeHeapSize() < 5000) {
    printf("Memory Full\n");
    ddTaskNode_t *node;
    while (completeList != NULL) {
        node = completeList;
        completeList = completeList->next;
        vPortFree(node);
    }
}

}

//-----
// Task List Management Functions
//-----
//Inserts new task into activeList, maintaining deadline order.
BaseType_t insertTask(ddTaskNode_t **activeList, ddTask_t newTask) { // create
new node and put the task inside -> to next task
    if (*activeList == NULL) {
        *activeList = (ddTaskNode_t *)pvPortMalloc(sizeof(ddTaskNode_t));
        (*activeList)->task = newTask;
        (*activeList)->next = NULL;

        //Create the task (userTask1()) -> Run the task
    }
}

```



```

        xTaskCreate(newTask.taskFunc, newTask.taskID,
configMINIMAL_STACK_SIZE, NULL, 0, &((*activeList)->task.taskHandle));
    } else {
        ddTaskNode_t *current = *activeList;
        ddTaskNode_t *previous = NULL;
        while (current != NULL && newTask.absDeadline >= current->
>task.absDeadline) {
            previous = current;
            current = current->next;
        }
        ddTaskNode_t *newNode = (ddTaskNode_t
*)pvPortMalloc(sizeof(ddTaskNode_t));
        newNode->task = newTask;
        newNode->next = current;
        xTaskCreate(newTask.taskFunc, newTask.taskID,
configMINIMAL_STACK_SIZE, NULL, 0, &newNode->task.taskHandle);
        if (previous != NULL)
            previous->next = newNode;
        else
            *activeList = newNode;
    }
    return pdTRUE;
}

//Removes the head of activeList (usually after completion or overdue) and
adds it to either complete or overdue list.
BaseType_t removeTask(ddTaskNode_t **activeList, ddTaskNode_t **retiredList) {
    if (*activeList == NULL)
        return pdFALSE;

    ddTaskNode_t *node = *activeList;
    *activeList = (*activeList)->next; // // Move the list pointer forward ->
to next task
    node->task.completionTime = getCurrentTime(); // // Save completion time
    printf("Task ID: %s, Release: %d, Complete: %d\n",
        node->task.taskID, node->task.releaseTime, node->
>task.completionTime); // and print to debug for the finished task

    if (node->task.taskHandle != NULL)
        vTaskDelete(node->task.taskHandle); // after that, delete the task
from memory. It's gone.

    node->next = *retiredList; // We move this task into the completeList for
record-keeping.
    *retiredList = node;
    return pdTRUE;
}

```

```

//Sets active task priority to the lowest (PAUSE_PRIO).
BaseType_t pauseScheduler(ddTaskNode_t **activeList) {
    if (*activeList != NULL) {
        vTaskPrioritySet((*activeList)->task.taskHandle, PAUSE_PRIO);
        return pdTRUE;
    }
    return pdFALSE;
}

//Sets the top-priority task in activeList to RUN_PRIO and resets deadline
timer.
BaseType_t updateScheduler(ddTaskNode_t **activeList) {
    if (*activeList != NULL) {
        //This sets the top task's priority to RUN_PRIO = 1. -> THIS is the
line that actually triggers the task to run.
        // the task was created before, but setting the priority here to
higher make it actually STARTS.
        vTaskPrioritySet((*activeList)->task.taskHandle, RUN_PRIO);
        uint32_t now = getCurrentTime();
        if ((*activeList)->task.absDeadline <= now) {
            xTimerChangePeriod(deadlineTimer, 1, 0);
        } else {
            xTimerChangePeriod(deadlineTimer, pdMS_TO_TICKS((*activeList)-
>task.absDeadline - now), 0);
        }
        xTimerReset(deadlineTimer, 0);
        // Starts or adjusts the deadlineTimer to expire when the task's
deadline is reached.
        // Which task is running, when to check for overdue, and time to run
the task
        return pdTRUE;
    } else {
        xTimerStop(deadlineTimer, 0);
        return pdFALSE;
    }
}

//-----
// RUN SECOND: Task Generator Task Releases periodic tasks at fixed intervals.
/*
This runs every TASK_GEN_INTERVAL ms and does:

Tracks how much time has passed (t1, t2, t3) for each task.

If t1 is divisible by TASK1_PERIOD, it's time to generate T1.

It builds a ddTask_t with:

```

```

taskFunc: pointer to userTask1

absDeadline: now + period

Other metadata

Sends this to DDS using releaseTask().
*/
//-----
void taskGeneratorTask(void *pvParameters) {
    uint32_t t1 = 0, t2 = 0, t3 = 0;
    uint32_t delta = TASK_GEN_INTERVAL;
    uint32_t now = 0;
    xSemaphoreGive(semRelease); // Initial trigger

    for (;;) {
        if (xSemaphoreTake(semRelease, portMAX_DELAY) == pdTRUE) {
            now = getCurrentTime();
            if ((t1 % TASK1_PERIOD) == 0) {
                uint32_t deadline = t1 + TASK1_PERIOD;
                ddTask_t task1 = {NULL, userTask1, PERIODIC, "T1", 1, now,
deadline, 0};
                releaseTask(task1); // -> Run first after releaseTask got
triggered
            }
            if ((t2 % TASK2_PERIOD) == 0) {
                uint32_t deadline = t2 + TASK2_PERIOD;
                ddTask_t task2 = {NULL, userTask2, PERIODIC, "T2", 2, now,
deadline, 0};
                releaseTask(task2);
            }
            if ((t3 % TASK3_PERIOD) == 0) {
                uint32_t deadline = t3 + TASK3_PERIOD;
                ddTask_t task3 = {NULL, userTask3, PERIODIC, "T3", 3, now,
deadline, 0};
                releaseTask(task3);
            }
            t1 += delta;
            t2 += delta;
            t3 += delta;
        }
    }
    vTaskDelete(NULL);
}

//-----
// Monitor Task Periodically reports counts of active, completed, and overdue
tasks.

```

```

/*
Triggered by semUpdate (via monitorTimer).

Gets all task lists using getActiveList, getCompletelist, and getOverdueList.

Prints current time and count of tasks in each category.
*/
//-----
void monitorTask(void *pvParameters) {
    ddTaskNode_t *lists[3] = {NULL, NULL, NULL}; // [0]: Active, [1]:
Complete, [2]: Overdue
    int counts[3] = {0, 0, 0};

    for (;;) {
        if (xSemaphoreTake(semUpdate, portMAX_DELAY) == pdTRUE) {
            getActiveList(&lists[0]);
            getCompleteList(&lists[1]);
            getOverdueList(&lists[2]);

            for (int i = 0; i < 3; i++) {
                counts[i] = 0;
                ddTaskNode_t *temp = lists[i];
                while (temp != NULL) {
                    counts[i]++;
                    temp = temp->next;
                }
            }
            printf("Time = %d | Active = %d | Complete = %d | Overdue =
%d\n\n",
                getCurrentTime(), counts[0], counts[1], counts[2]);
        }
    }
}

//-----
// DDS Interface Functions
//-----
BaseType_t releaseTask(ddTask_t task) {
    DDSMessage_t msg = {RELEASE_EVENT, task};
    return xQueueSend(requestQueue, &msg, portMAX_DELAY);
}

BaseType_t completeTask(void) {
    DDSMessage_t msg = {COMPLETE_EVENT, {0}}; // Sends a COMPLETE_EVENT
message to DDS.
    return xQueueSend(requestQueue, &msg, portMAX_DELAY); // DDS will wake up
from its xQueueReceive() call.
    // -> back to deadlineSchedulerTask

```

```

}

void getTaskLists(ddTaskNode_t **active, ddTaskNode_t **overdue, ddTaskNode_t
**complete) {
    if (uxQueueSpacesAvailable(outputQueue) < QUEUE_LEN)
        xQueueReset(outputQueue);
    DDSMessage_t req = {GET_LISTS, {0}};
    xQueueSend(requestQueue, &req, portMAX_DELAY);
    xQueueReceive(outputQueue, active, portMAX_DELAY);
    xQueueReceive(outputQueue, overdue, portMAX_DELAY);
    xQueueReceive(outputQueue, complete, portMAX_DELAY);
}

void getActiveList(ddTaskNode_t **active) {
    if (uxQueueSpacesAvailable(outputQueue) < QUEUE_LEN)
        xQueueReset(outputQueue);
    DDSMessage_t req = {GET_ACTIVE, {0}};
    xQueueSend(requestQueue, &req, portMAX_DELAY);
    xQueueReceive(outputQueue, active, portMAX_DELAY);
}

void getCompleteList(ddTaskNode_t **complete) {
    if (uxQueueSpacesAvailable(outputQueue) < QUEUE_LEN)
        xQueueReset(outputQueue);
    DDSMessage_t req = {GET_COMPLETE, {0}};
    xQueueSend(requestQueue, &req, portMAX_DELAY);
    xQueueReceive(outputQueue, complete, portMAX_DELAY);
}

void getOverdueList(ddTaskNode_t **overdue) {
    if (uxQueueSpacesAvailable(outputQueue) < QUEUE_LEN)
        xQueueReset(outputQueue);
    DDSMessage_t req = {GET_OVERDUE, {0}};
    xQueueSend(requestQueue, &req, portMAX_DELAY);
    xQueueReceive(outputQueue, overdue, portMAX_DELAY);
}

//-----
// User Task Functions Simulate user-defined periodic tasks with specific
// execution times.
//-----
void userTask1(void *pvParameters) {
    delayMS(TASK1_EXEC); // simulate doing the task by giving it delay
    completeTask(); // alarm DDS that the task is finished
    for (;;) ; // wait for response in the infinite loop
}

void userTask2(void *pvParameters) {

```

```

    delayMS(TASK2_EXEC);
    completeTask();
    for (;;) ;
}

void userTask3(void *pvParameters) {
    delayMS(TASK3_EXEC);
    completeTask();
    for (;;) ;
}

//-----
// Timer Callback Functions
//-----

//Sends OVERDUE_EVENT to DDS.
void deadlineTimerCallback(TimerHandle_t xTimer) {
    DDSMessage_t msg = {OVERDUE_EVENT, {0}};
    BaseType_t highPrio = pdFALSE;
    xQueueSendToFrontFromISR(requestQueue, &msg, &highPrio);
    if (highPrio)
        taskYIELD();
}

//Increments global system time by updating the TimerID
void systemTimerCallback(TimerHandle_t xTimer) {
    uint32_t curTime = (uint32_t)pvTimerGetTimerID(xTimer);
    curTime++;
    vTimerSetTimerID(xTimer, (void *)curTime);
}

//Triggers either task generation or monitoring depending on the timer ID.
void referenceTimerCallback(TimerHandle_t xTimer) {
    BaseType_t highPrio = pdFALSE;
    uint32_t timerID = (uint32_t)pvTimerGetTimerID(xTimer);
    if (timerID == 2) {
        xSemaphoreGiveFromISR(semRelease, &highPrio);
    } else {
        xSemaphoreGiveFromISR(semUpdate, &highPrio);
    }
    if (highPrio)
        taskYIELD();
}

//-----
// Hardware and Utility Functions
//-----
uint32_t getCurrentTime(void) {

```

```

    return (uint32_t)pvTimerGetTimerID(sysTimer);
}

void delayMS(uint32_t ms) {
    uint32_t start;
    for (; ms > 0; ms--) {
        start = xTaskGetTickCount();
        while ((xTaskGetTickCount() - start) == 0)
            ;
    }
}

static void setupHardware(void) {
    NVIC_SetPriorityGrouping(0);
}

//-----
// FreeRTOS Hook Functions
//-----
void vApplicationMallocFailedHook(void) {
    printf("SYSTEM OUT OF MEMORY");
    for (;;) ;
}

void vApplicationStackOverflowHook(xTaskHandle pxTask, signed char
*pcTaskName) {
    (void)pxTask;
    (void)pcTaskName;
    for (;;) ;
}

void vApplicationIdleHook(void) {
    volatile size_t freeHeap = xPortGetFreeHeapSize();
    if (freeHeap > 100) {
        // Optionally adjust configuration if needed.
    }
}

```